



Naan Mudhalvan

Industrial IoT and Industry 4.0

Tektork Private Limited, Coimbatore

IoT Protocols - MQTT

Introduction to MQTT

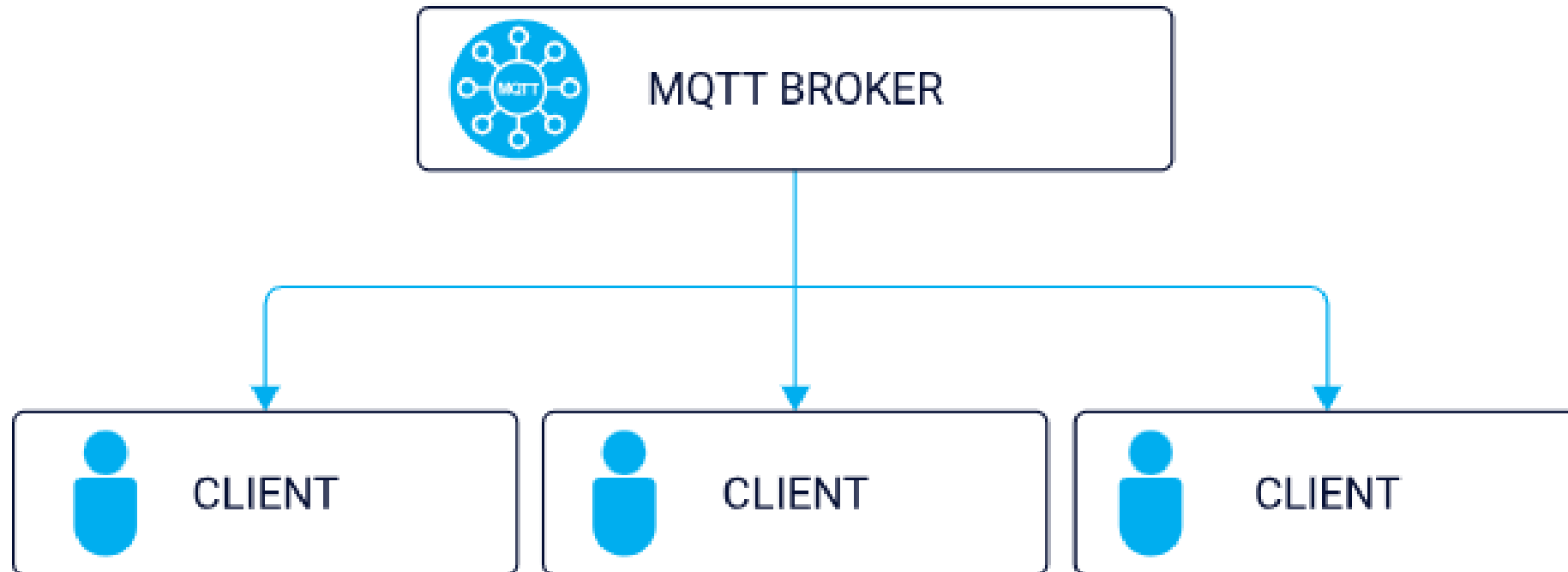
- **MQTT (Message Queuing Telemetry Transport)** is a lightweight, simple messaging protocol designed for devices with limited resources and low bandwidth.
- It is especially suited for **Internet of Things (IoT)** applications, enabling reliable **machine-to-machine (M2M)** communication in environments with constrained network capacity, high latency, or intermittent connectivity.
- MQTT operates on a **client-server model**, where multiple clients (devices or applications) connect to a central **broker** over a TCP-based connection.
- Clients can act as **publishers** by sending messages to specific **topics**, or as **subscribers** by registering interest in those topics to receive relevant messages.

Introduction to MQTT Cond..

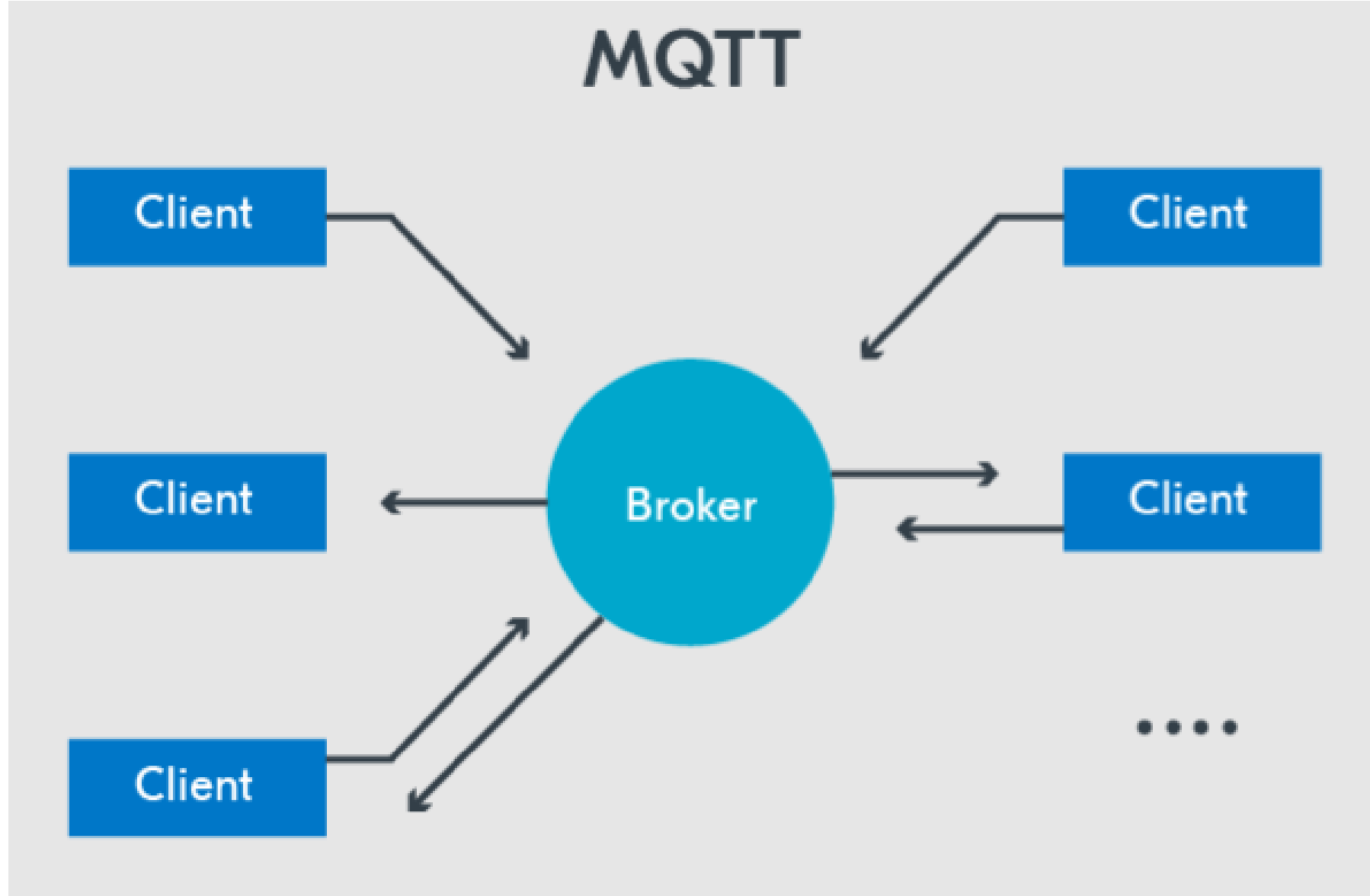
- The broker is responsible for routing messages between publishers and subscribers, ensuring that messages are delivered reliably according to configurable **Quality of Service (QoS)** levels.
- The protocol's **publish-subscribe** communication model decouples message producers and consumers, allowing flexible and scalable communication patterns ideal for IoT ecosystems.



MQTT Architecture



MQTT Architecture



MQTT Architecture

- An **MQTT connection** is always established between the **client** and the **broker**.
- As mentioned earlier, MQTT separates the **publisher** from the **subscriber**, and clients never connect to each other directly.
- All communication between clients is handled exclusively by the **broker**.
- To initiate a connection, the client sends a CONNECT message to the broker.
- The broker responds with a CONNACK message containing a status code.
- Once the connection is successfully established, the broker keeps it open until the client sends a DISCONNECT message or the connection is otherwise terminated.

MQTT Architecture Cond..

- MQTT supports the ability to **retain messages** on the broker.
- This behavior is controlled by a **retain flag** set when publishing the message.
- A retained message is stored on the broker and immediately sent to any client that subscribes to the corresponding topic.
- This allows new subscribers to receive the most recent message without having to wait for the next publication.
- Without this feature, a newly subscribed client might have to wait hours or even days before receiving relevant data.

MQTT Architecture Cond..

- MQTT also includes a feature called **Last Will and Testament (LWT)**.
- The LWT message is specified by the client during the connection setup and includes a topic, QoS level, and the message payload.
- If a client disconnects unexpectedly—due to a timeout, I/O error, or power failure—the broker automatically publishes the LWT message to the specified topic.
- This allows other clients to be notified of the unexpected disconnection.

MQTT Architecture Cond..

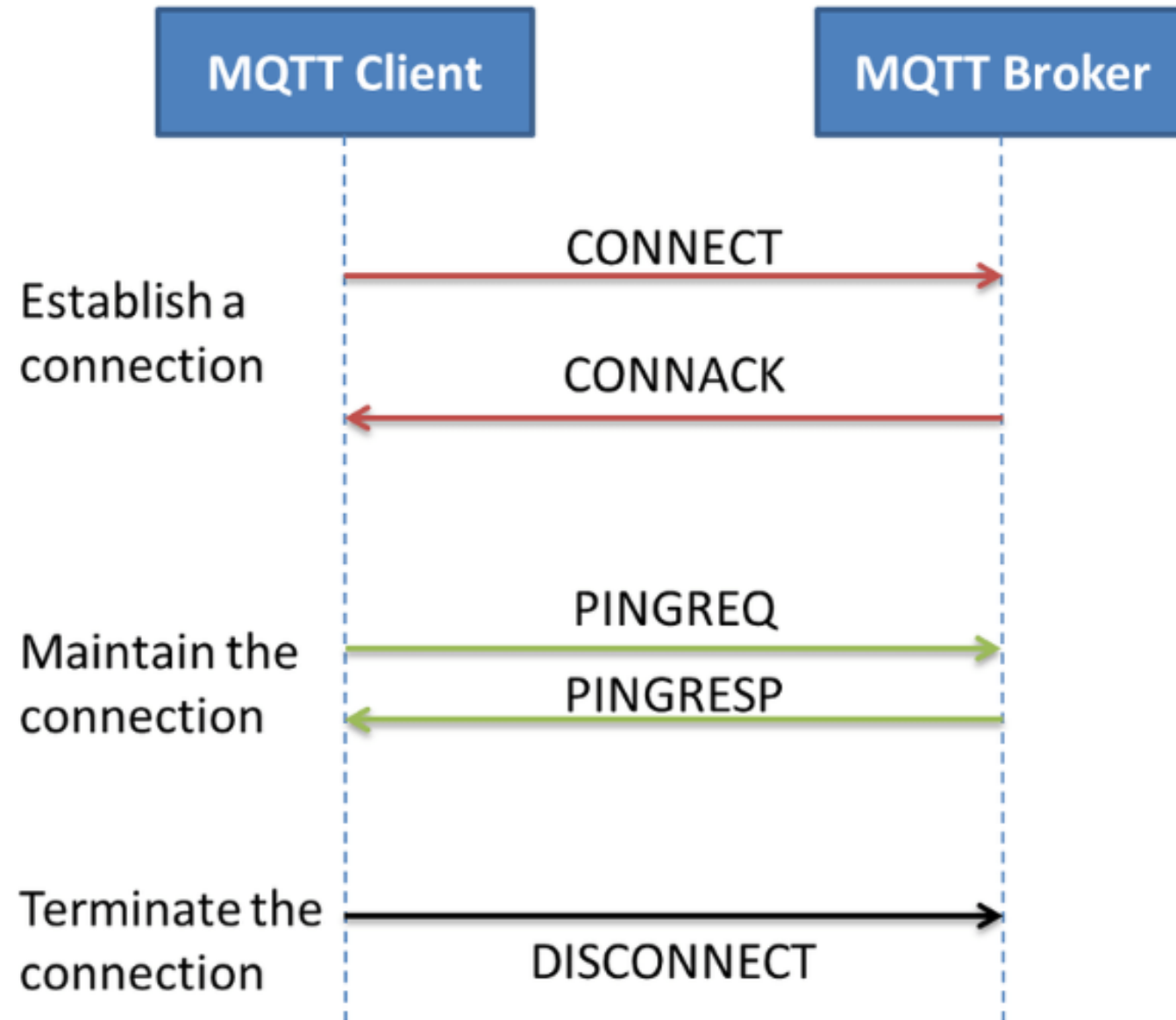
- Although MQTT runs over **TCP**, which is a reliable transport protocol, connections can still drop—particularly with wireless or battery-powered devices.
- In such cases, the session may enter a **half-open state**, where one side believes the connection is still active while the other does not.
- To avoid this, MQTT uses a **keep-alive mechanism**.
- During the connection, the client specifies a keep-alive interval.
- If no data is transmitted during this time, the client sends a PINGREQ packet to the broker.

MQTT Architecture Cond..

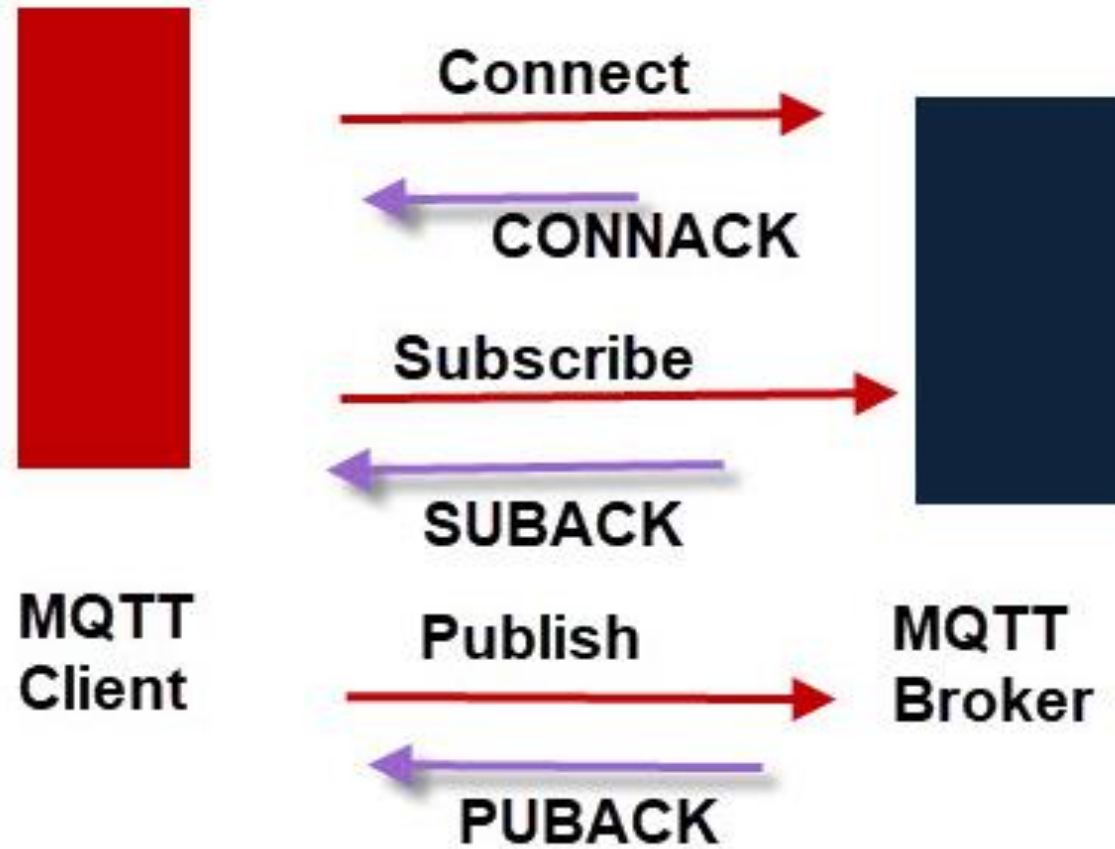
- The broker must respond with a PINGRESP to confirm that the connection is still active.
- If the broker does not receive any packets from the client within **1.5 times the keep-alive interval**, it will assume the connection has failed.
- The broker will then close the connection and, if an LWT message was set, publish it to notify all subscribed clients.

Component	Role	Description
Broker	Server / Message Hub	The central nervous system. It receives all messages, filters them by topic, and distributes them to subscribed clients. It is responsible for managing connections, sessions, and subscriptions.
Client	Publisher or Subscriber	Any device or application that connects to the broker. A client can be both a publisher and a subscriber. Examples: sensors, mobile apps, actuators, servers.
Topic	Address / Routing Key	A hierarchical UTF-8 string that the broker uses to filter messages for each client (e.g., sensor/floor3/room207/temperature).
Message	The Data Payload	The actual information being carried, consisting of a topic (address) and a payload (data).

MQTT Architecture Cond..



MQTT Message Flow



Note:

MQTT operates as a command-response protocol, meaning every command sent by a client (such as connect, publish, or subscribe) is acknowledged by the broker. Clients **must establish a connection** with the broker before they are allowed to **publish** data or **subscribe** to topics.

Broker

- **Broker** acts as a mediator between **publishers** (clients/devices sending messages) and **subscribers** (clients/devices receiving messages).
- The broker manages **topics** or **queues** where messages are stored temporarily until subscribers receive them.
- The **FIFO** pattern ensures that messages are delivered in the order they were received, which is crucial for many applications.
- The broker also often handles things like **message durability** (whether messages persist through broker restarts), **QoS levels** (quality of service for message delivery), and **client sessions** (maintaining state for connected clients).

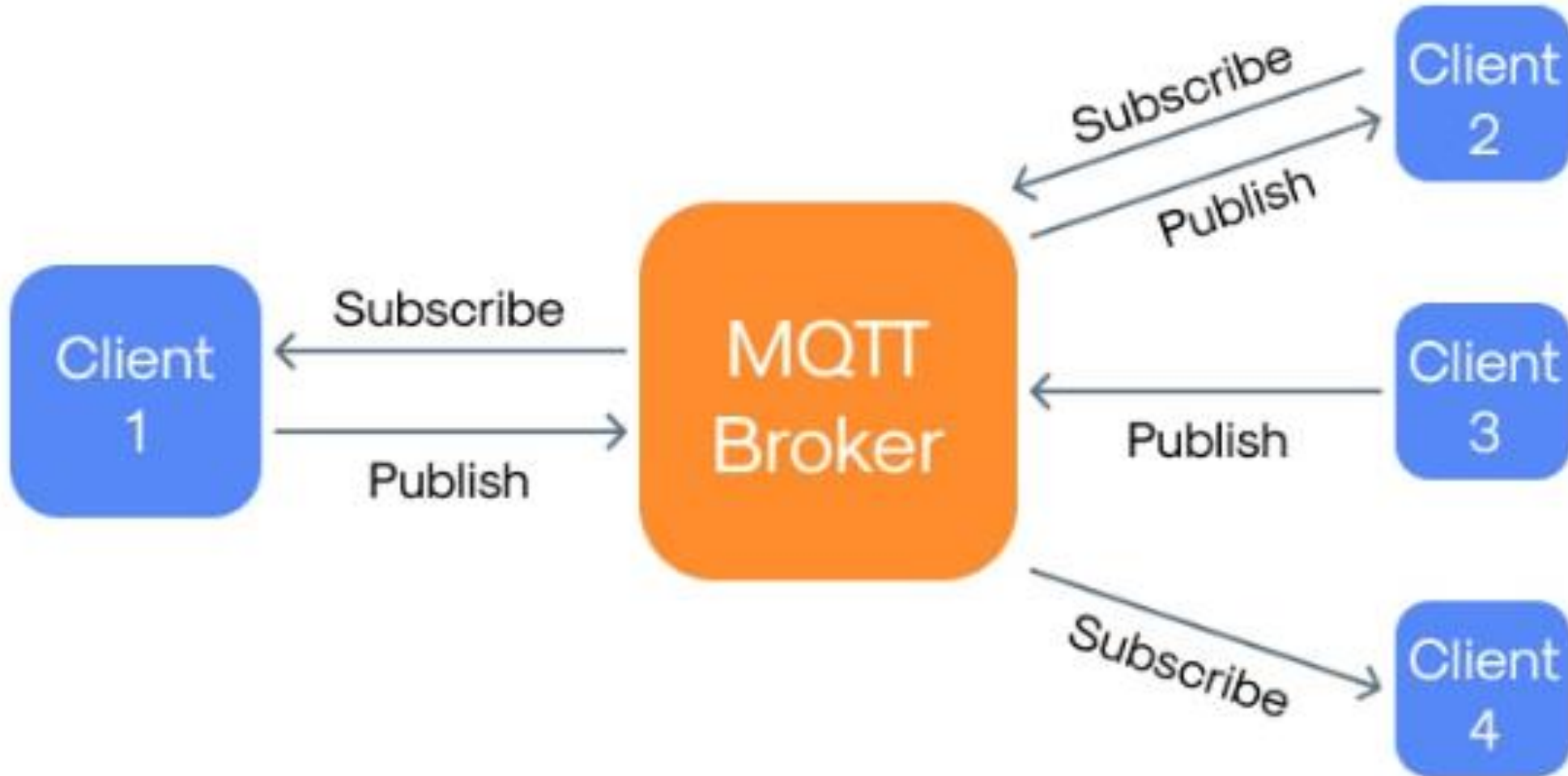
Broker

- Authentication and authorization ensure that only valid clients can connect and access specific topics or queues, which is important for security.
- The **broker** is responsible for receiving all messages, filtering them, identifying subscribers for each message, and delivering them to the appropriate clients.
- It also stores session data for all connected clients, including subscriptions and any missed messages.
- Most brokers are designed to be extensible, allowing easy integration with backend systems for user management and message processing.

Broker Cond..

- This integration is especially important because the broker is often the component directly exposed to the internet, serving many clients and forwarding data to analytics or processing systems.
- Since all messages pass through the broker, it must be **scalable, integrated with backend infrastructure, easy to monitor, and fault-tolerant**.
- One of the most popular open-source MQTT brokers is **Eclipse Mosquitto**.
- It is a lightweight MQTT message broker that implements MQTT protocol versions **5.0, 3.1.1, and 3.1**.

Broker



Popular Brokers

You need a broker to use MQTT. Popular choices include:

- **Eclipse Mosquitto:** A very lightweight, open-source broker, perfect for testing and small deployments.
- **EMQX:** A highly scalable, open-source broker built for large clusters.
- **HiveMQ:** A popular enterprise-grade commercial broker.
- **AWS IoT Core / Azure IoT Hub:** Cloud-managed MQTT broker services that integrate deeply with their respective cloud platforms.

MQTT Clients: Publishers and Subscribers

- **MQTT clients** can act as both **publishers** and **subscribers**.
- They are referred to as publishers or subscribers depending on their role at a given moment—either **publishing messages** or **subscribing to receive them**.
- A single MQTT client can implement the functionality of both a publisher and a subscriber at the same time.
- An MQTT client is **any device**, from a **microcontroller** to a **full-featured server**, that uses an MQTT library to connect to a broker over a network.



MQTT Clients: Publishers and Subscribers

For example:

- An MQTT client can be a **small, resource-constrained device** using a minimal MQTT library over a wireless connection.
- It can also be a **desktop or laptop computer** using a graphical MQTT client tool for testing or development purposes.

MQTT Clients: Publishers and Subscribers Cond..

- The MQTT client implementation is **simple and lightweight**, making it ideal for use in **small devices**. MQTT client libraries are available for **various programming languages and platforms**.
- There are many **C/C++ MQTT implementations** designed specifically for **IoT devices**.

For example:

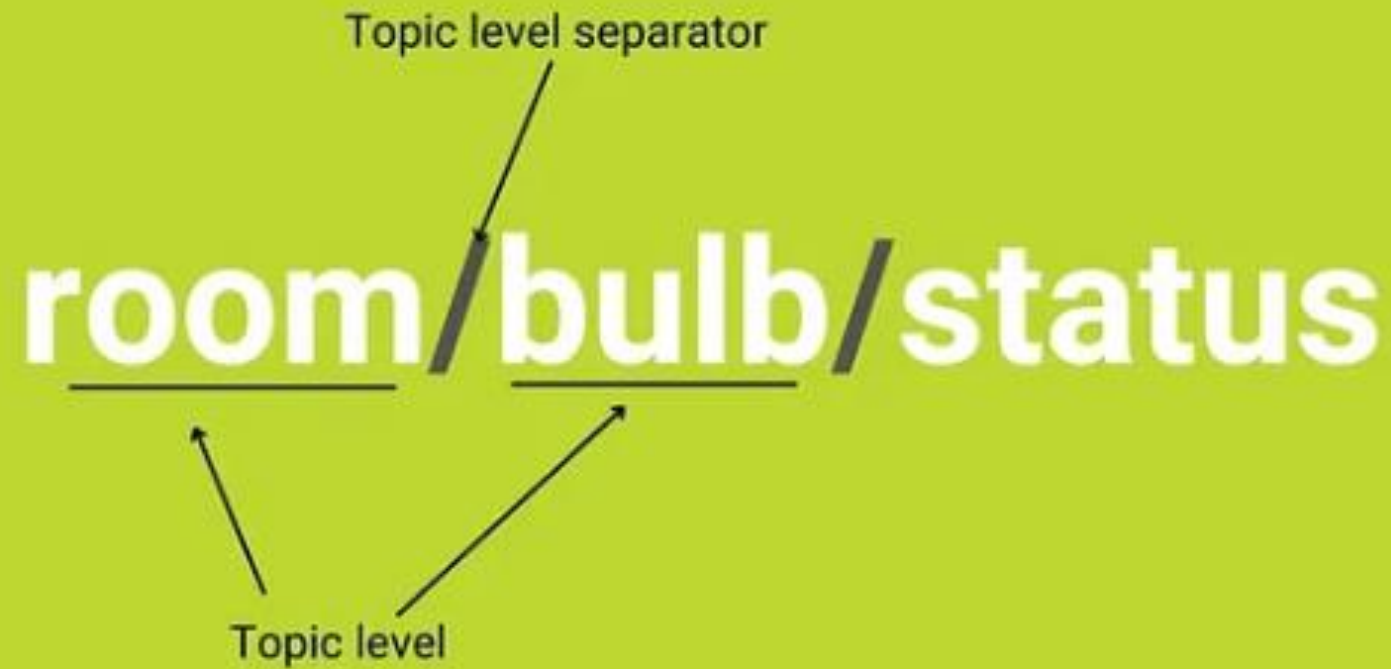
- The **LWIP (Lightweight TCP/IP)** stack includes an MQTT client written in C.
- Another widely used library is **Eclipse Paho**, which supports multiple languages including C.

MQTT topic

- An **MQTT topic** is a **UTF-8 encoded string** used by the broker for **message filtering and routing**.
- Topics are represented as **hierarchical strings** separated by **forward slashes (/)**. Clients **publish messages** to specific topics, and subscribers **register interest** in those topics to receive matching messages.
- The broker **handles routing** messages from publishers to the appropriate subscribed clients based on **topic filtering**.
 - Example

home/livingroom/temperature

MQTT topic Structure



Topic Structure

Three Type of Filters

■ Topic filters

- Topic filters allow clients to subscribe to specific topics or branches of topics, ensuring they only receive the data they are interested in.
- Each published message must include a topic, and the broker is responsible for delivering the message to the appropriate subscribers or discarding it if there are no matching subscriptions.

Three Type of Filters Cond..

■ Content filters

- Content filters enable brokers to inspect and filter the content of published messages.
- If the data is not encrypted, the broker can process, block, or modify the message before storing or forwarding it to subscribers.

■ Type filters

- Type filters are applied by clients that process incoming data streams.
- Clients can analyze the content of messages they receive and decide whether to process or ignore them based on the message type or structure.

Three Type of Filters Summary

Filter Type	How It Works	Who Performs It	Key Benefit	Example
1. Topic Filter	Uses a topic string (e.g., sensor/temp) or wildcards (+, #) to select messages.	The Broker	Most efficient. Prevents unwanted data from being sent over the network to the client.	Subscribe to sensor/+ /temp to get temperature from all sensors.
2. Content Filter	Inspects the payload (content) of a message against a set of rules.	The Broker (via extensions)	Offloads processing. Applies complex logic at the broker before any client sees the data.	"Forward messages from sensor/temp to the alarm topic only if the payload value is > 30."
3. Client-Side Filter	The client's application code inspects received messages and decides to process or ignore them.	The Subscribing Client	Maximum flexibility. The client can apply its own unique logic to the data it receives.	A dashboard receives all sensor/status messages but only displays devices with a status of "ERROR".

MQTT Message

An **MQTT message**, also known as an **MQTT control packet**, is the basic unit of data transferred between an MQTT client and a broker. It's a binary-encoded packet with a structured format, designed to be as small as possible to minimize network overhead.

Message Structure

Every MQTT message is composed of up to three parts:

- **Fixed Header (required)**
- **Variable Header (optional):**
- **Payload (optional):**

MQTT Message

Fixed Header (required):

This is the first part of every MQTT message. It's at least two bytes long and contains essential information for processing the packet.

- **Control Packet Type (4 bits):** Specifies the type of MQTT message, such as CONNECT, PUBLISH, SUBSCRIBE, PUBACK, etc. There are 14 different packet types defined in the protocol.
- **Flags (4 bits):** These bits are used to carry additional control information, such as the **Quality of Service (QoS)** level, the DUP (Duplicate) flag, and the RETAIN flag. The meaning of these flags can vary depending on the message type.
- **Remaining Length (variable bytes):** Indicates the size of the rest of the message (the variable header and payload). It uses a variable-length encoding scheme to keep the message compact.

MQTT Message

Variable Header (optional):

This section, when present, contains message-specific metadata. Its contents depend on the control packet type. For example:

- A PUBLISH message's variable header includes the **topic name** and a **Packet Identifier**.
- A SUBSCRIBE message's variable header contains a **Packet Identifier**.
- A CONNECT message's variable header includes the protocol name, version, and connection flags.

MQTT Message

Payload (optional):

This is the actual data or content of the message. The payload's format is arbitrary and can be anything from simple text to JSON, binary data, or encrypted information. The content of the payload depends entirely on the message type.

- For a **PUBLISH** message, the payload is the data being sent.
- For a **SUBSCRIBE** message, the payload is a list of topics the client wants to subscribe to, along with the desired QoS for each.

Key Message Types

MQTT defines several message types, each with a specific purpose in the client-broker communication flow:

- **CONNECT:** A client sends this message to establish a connection with the broker.
- **CONNACK:** The broker's acknowledgment to a CONNECT message, indicating whether the connection was successful.
- **PUBLISH:** Used by a client to send application data to a topic.
- **SUBSCRIBE:** A client sends this to express interest in receiving messages on one or more topics.
- **PUBACK/PUBREC/PUBREL/PUBCOMP:** These messages are part of the multi-step handshake for higher QoS levels, ensuring reliable message delivery.
- **PINGREQ/PINGRESP:** Used to confirm that the network connection between the client and broker is still alive.
- **DISCONNECT:** Sent by a client to gracefully close its connection.

Key Message Types Summary

Message Type	Direction	Purpose
CONNECT	Client → Broker	Initiates a connection to the broker. Contains Client ID, will message, keep alive timer, and credentials.
CONNACK	Broker → Client	Acknowledges the connection request. Contains a return code (success or reason for failure).
SUBSCRIBE	Client → Broker	Requests a subscription to one or more topic filters.
SUBACK	Broker → Client	Acknowledges the SUBSCRIBE request and confirms the granted QoS for each topic.
UNSUBSCRIBE	Client → Broker	Requests to unsubscribe from one or more topics.
UNSUBACK	Broker → Client	Acknowledges the UNSUBSCRIBE request.
PINGREQ	Client → Broker	Sent to keep the connection alive if no other messages are being sent.
PINGRESP	Broker → Client	Responds to a PINGREQ, confirming the connection is alive.
DISCONNECT	Client → Broker	Gracefully terminates the connection. The broker will then publish the client's Will Message if it disconnected unexpectedly.

MQTT Quality of Service (QoS) Level

QoS 0 – At Most Once (Unconfirmed Transmission)

- This is the **lowest** level of service.
- Also known as "**fire-and-forget**", it provides **no acknowledgment** of message receipt.
- The message is **sent once**, and the sender does **not retry** if the message is lost.
- It is the **most efficient** in terms of bandwidth and processing, but there is **no guarantee** that the message reaches the recipient.

MQTT Quality of Service (QoS) Level cond..

QoS 1 – At Least Once (Guaranteed Delivery)

- This level **guarantees** that the message is delivered **at least once** to the recipient.
- However, **duplicates may occur**.
- After receiving the message, the subscriber sends a **PUBACK** (publish acknowledgment) to the sender.
- If the sender does not receive PUBACK, it **retransmits** the message.

MQTT Quality of Service (QoS) Level

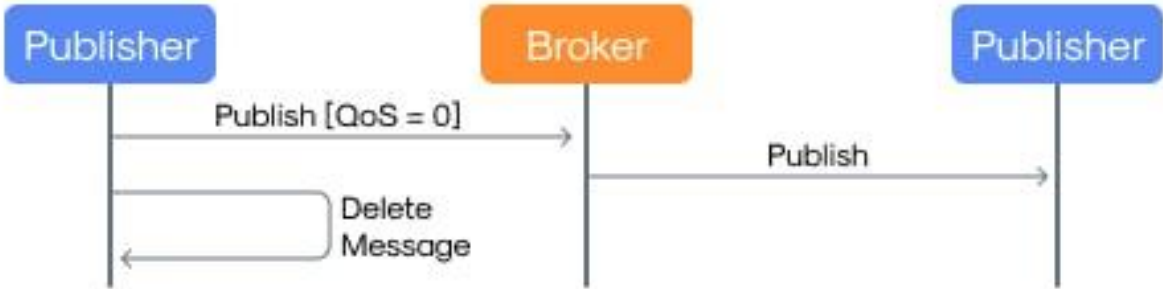
QoS 2 – Exactly Once (Guaranteed Service)

- This is the **highest level of QoS**, ensuring that the message is delivered **exactly once**, with **no duplication**.
- It uses a **four-step handshake** to confirm successful delivery:
 - The sender sends a PUBLISH message.
 - The receiver replies with PUBREC (publish received).
 - The sender responds with PUBREL (publish release).
 - The receiver completes the process with PUBCOMP (publish complete).
- Until the PUBCOMP message is exchanged, the receiver **caches the original message** to prevent reprocessing in case of retransmission.

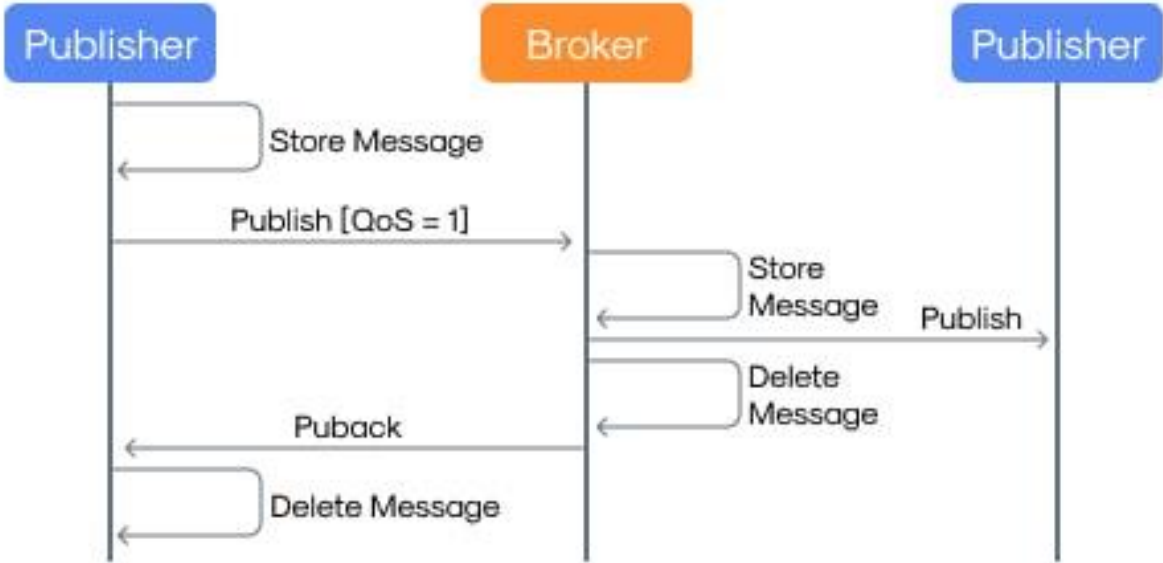


MQTT Qos Levels

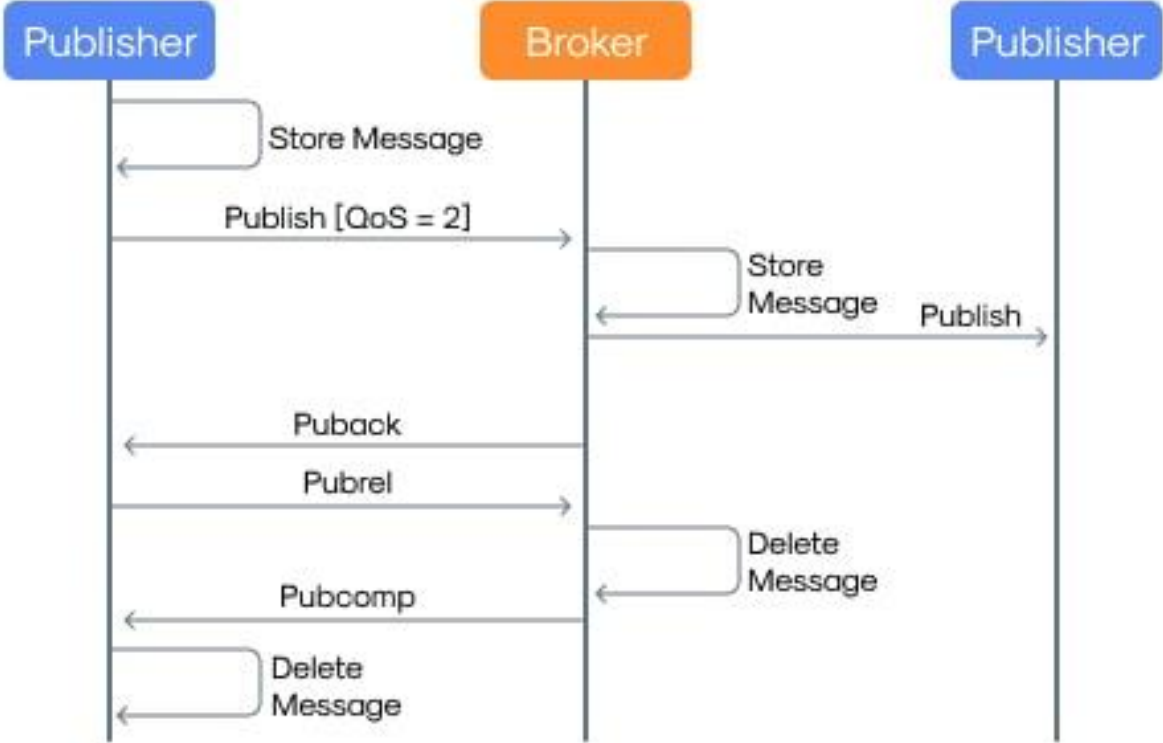
QoS 0: At Most Once (Fire And Forget)



QoS 1: At Least Once



Qos 2: Exactly Once



MQTT QoS Level Summary

QoS Level	Name	Delivery Guarantee	Packet Flow	Description	Use Case Example
0	At most once	Fire and forget. A message is delivered at most once. It may be lost.	PUBLISH → (No response)	The fastest but least reliable method. No retransmission or acknowledgment is used.	Non-critical sensor data (e.g., ambient temperature readings where a missed value is acceptable).
1	At least once	A message is guaranteed to be delivered at least once. Duplicates are possible.	PUBLISH → PUBACK	The sender stores the message until it receives a PUBACK from the receiver. If no ACK is received, it resends.	Command and control (e.g., "Turn on the pump," where duplication is better than the command being lost).
2	Exactly once	The highest assurance. A message is guaranteed to be delivered exactly once.	PUBLISH → PUBREC → PUBREL → PUBCOMP	A four-step handshake ensures no duplication. This is the most reliable but also the slowest method.	Critical commands or transactions (e.g., "Process payment," where duplication would cause a significant problem).

Advantages of MQTT Protocol

Its ease of use

- MQTT's ease of use is one of its most compelling features for IoT. It offers ready-made clients and brokers, allowing for quick and straightforward implementation.

It is reliable

- MQTT queues messages and retries delivery, effectively handling intermittent connections that are common in IoT environments. Its Quality of Service (QoS) levels add another layer of reliability, ensuring that messages are delivered accordingly.

Advantages of MQTT Protocol Cond..

It is bidirectional

- MQTT supports omnidirectional communication. This means any device in the network can act as a publisher or a subscriber to any topic.

It is scalable and future-proof

- MQTT handles broadcasting a message to a hundred devices or a million effortlessly, the process remains the same – publish to a common topic, and all subscribed devices receive the message.

Feature	Description	Benefit for IoT
Pub/Sub Model	Decouples producers and consumers of data.	Devices don't need to know each other; highly scalable.
Lightweight	Small packet size, minimal overhead.	Saves battery life and bandwidth.
Quality of Service (QoS)	Multiple levels of delivery guarantee.	Adapts to unreliable networks.
Persistent Sessions	Broker stores state for disconnected clients.	No data is lost if a device goes offline.
Last Will & Testament	Alert other devices if a client disconnects abruptly.	Enables failure detection and notification.
Retained Messages	Get the last known value immediately on subscribe.	Devices get the latest state instantly on startup.

MQTT Work Flow

- When a client connects to a broker, it must first identify itself using the **MQTT CONNECT** packet. This packet contains the client's identifier, the broker's address, and any necessary security credentials (such as username and password).
- Once connected, the client can start **publishing** and **subscribing** to messages.
- To **publish** a message, the client sends an **MQTT PUBLISH** packet. This packet includes the topic name, the message payload, and the Quality of Service (QoS) level for the message delivery.
- To **subscribe** to a topic, the client sends an **MQTT SUBSCRIBE** packet, specifying the topic or topics it wants to receive messages from.

MQTT Work Flow Cond..

- When a client publishes a message, the broker forwards the message to all clients that have subscribed to the corresponding topic.
- Similarly, when a client subscribes to a topic, the broker begins delivering all messages published to that topic to the subscriber.

MQTT Applications and Real Time usage.

Applications use message queueing telemetry transport (MQTT) to function are:

- **Smart Cities** : Smart cities which include (smart transport, smart healthcare, smart school, smart meter and smart home) required to set up a quick, faster and standard communication protocol like MQTT.
- **Remote area Sensors** : Sensor based devices / application installed at **remote** area which regularly sends the messages to its server required to run at low bandwidth internet and low cost infrastructure setup. The MQTT protocol suits these types of applications.
- **Wearable Devices** : To make data transmission in low energy wearable devices like smart watches, smart shoes etc.

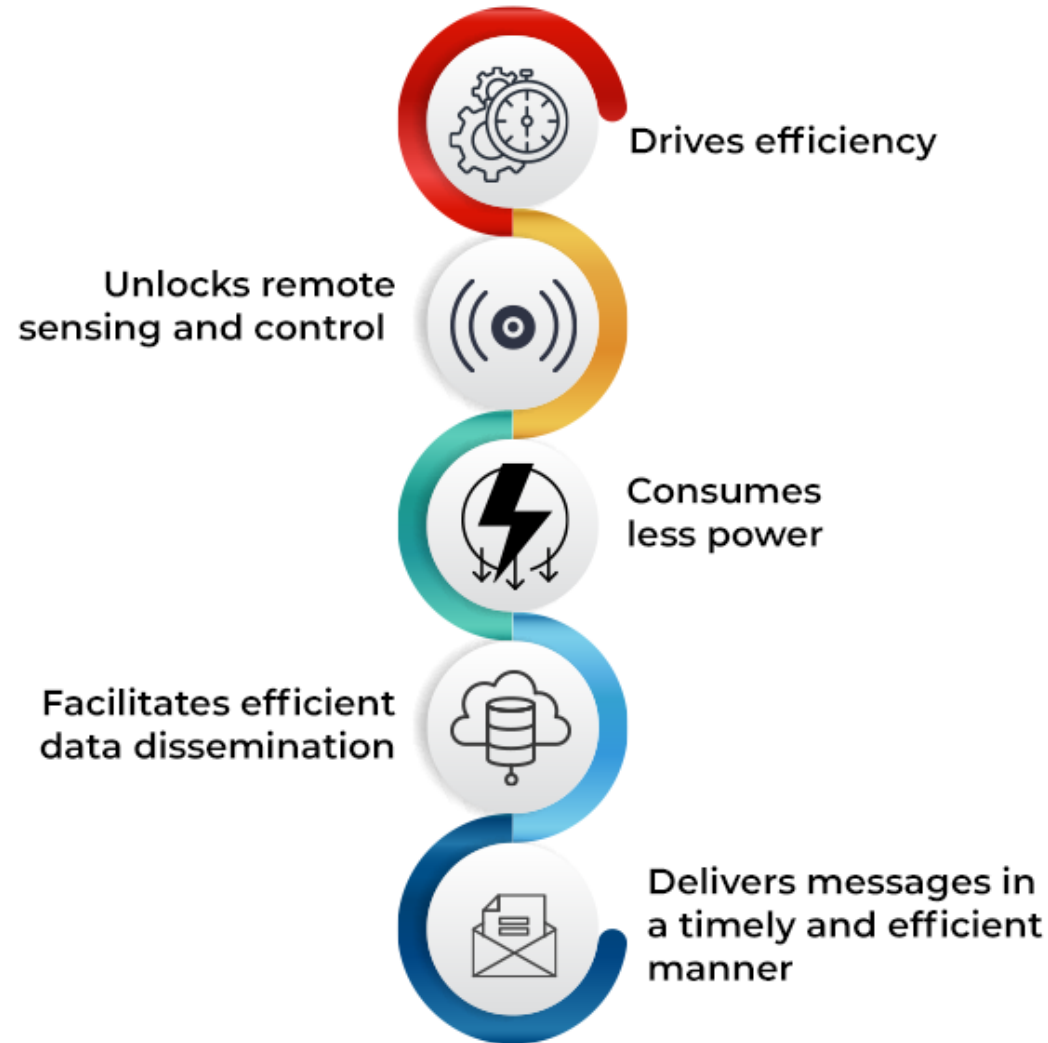
MQTT Applications and Real Time usage Cond..

- **Social Media Applications** : Facebook using MQTT protocol for transmission for its Instagram and Messenger applications. It takes low bandwidth and battery of the device.
- **Smart Farming** : Real time information of weather, sunlight, soil pH, moisture, humidity we can get from different MQTT enabled IoT devices, these information can make farming effective.
- **Manufacturing** : IoT devices have various sensors that can monitor the temperature, vibration, voltage, currents, and more. So, these MQTT enabled sensors can help with early warning signs of any predicted malfunctions, or downtime.
- **Smart Home Automation** : MQTT enabled home devices can enhance the energy efficiency, security and cost savings of the home. IoT enabled smart devices can send data to broker and data on the broker can be read by any mobile application.

Importance of MQTT in IoT



IMPORTANCE OF MQTT IN IOT



Importance of MQTT in IoT

Drives Efficiency

- MQTT is a lightweight protocol, making it both efficient and quick to implement. Its architecture allows it to run smoothly even on devices with low-power CPUs and limited RAM.
- This efficiency leads to faster data transmission compared to heavier protocols like AMQP.
- Thanks to its lightweight design, MQTT can be rapidly and easily deployed across various applications.

Importance of MQTT in IoT Cond..

Enables Low Network Usage

- Because MQTT uses smaller data packets, it minimizes network bandwidth consumption.
- This is particularly important in environments where sensor data is transmitted wirelessly over Low Power Wide Area Network (LPWAN) systems.
- While individual packets are small, the total volume of data from many sensors can be substantial.
- MQTT efficiently handles this data flow, delivering information to analytics and visualization tools, whether hosted in the cloud or on-premises.

Importance of MQTT in IoT Cond..

Facilitates Efficient Data Dissemination

- An architecture that uses multiple MQTT brokers positioned at the network edges is very effective for distributing large amounts of edge-generated data.
- This edge-based approach reduces latency and decreases the resource consumption on central cloud servers.
- However, managing heterogeneity across different network edges can be challenging, as MQTT broker implementations may vary depending on the specific environment.

Importance of MQTT in IoT Cond..

Enables Remote Sensing and Control

- MQTT is a simple publish-subscribe protocol built on top of TCP/IP to ensure reliable data delivery with minimal loss.
- The name MQTT stands for **Message Queuing Telemetry Transport**—a protocol designed to transport various types of data.
- Its main goal is to enable communication with remote devices where network capacity is limited and only minimal code is required.

Importance of MQTT in IoT Cond..

Delivers Messages Timely and Efficiently

- MQTT is designed to handle complex scenarios requiring reliable message delivery. Clients can specify different **Quality of Service (QoS)** levels to ensure messages are delivered as expected.
- MQTT brokers are required to support these QoS levels, which act as agreements or contracts between the broker and clients.
- When both parties agree on a QoS level, the broker is responsible for honoring that agreement to guarantee message delivery.

Importance of MQTT in IoT Cond..

Consumes Less Power

- MQTT is highly beneficial for connected devices as it optimizes network bandwidth and reduces power consumption.
- Because MQTT is a binary protocol, it has low overhead. It maintains a constant TCP connection between the client and the broker, which eliminates the need to establish a new connection for each published message, thereby saving time and energy.
- A significant amount of power is saved when a client subscribes to messages and waits for data from a remote network.
- Unlike plain HTTP, which requires frequent polling and consumes a lot of power—even when there are no new messages—MQTT uses an open TCP connection that allows the broker to **push** messages directly to the client.
- This approach is much more power-efficient and was one of the core reasons MQTT was designed.

Importance of MQTT in IoT Cond..

Supports Multiple Modes of Communication

- MQTT uses a **publish-subscribe** communication model, where publishers send messages to topics, and subscribers receive messages for topics they are interested in.
- There is no direct connection between clients; instead, all communication is routed through the broker.

MQTT supports various communication modes, including:

- **One-to-one** communication
- **Bidirectional** communication
- **One-to-many** communication

This flexibility makes MQTT suitable for a wide range of IoT applications.

Understanding MQTT Security

1. Authentication

- MQTT supports client authentication to verify the identity of devices connecting to the broker. Typically, this is done using **username and password** credentials.
- For stronger security, advanced implementations may use **client certificates** or **tokens**, which provide more robust authentication mechanisms.

2. Authorization

- Once a client is authenticated, MQTT brokers enforce **authorization** to control what resources the client can access and what actions it can perform.
- This is usually managed by configuring **topic permissions** on the broker, restricting clients to publish or subscribe only to authorized topics.

Understanding MQTT Security

3. Transport Layer Security (TLS/SSL)

- To protect data during transmission, MQTT supports **TLS/SSL encryption** at the transport layer.
- This encrypts all data exchanged between clients and the broker, preventing eavesdropping and tampering. TLS/SSL also enables mutual authentication, ensuring both client and server verify each other's identity.

4. Payload Encryption

- Beyond transport layer encryption, MQTT allows for **payload encryption**.
- This means the actual message content (payload) can be encrypted independently, providing an extra layer of security.
- Payload encryption is especially valuable when full transport encryption is not feasible or when protecting highly sensitive data is necessary.

Thank You